

Task Queue with Dependencies

You are building a task scheduling system for a distributed computing platform. Tasks have priorities, dependencies on other tasks, and must be executed in a valid order that respects all dependencies. The system must support adding tasks, querying task status, and retrieving the next executable task based on priority while ensuring all dependencies are satisfied.

Implement a class `TaskQueue` that manages a collection of tasks with dependencies. It should support:

- **`add_task(task_id, priority, dependencies)`**, where `dependencies` is a list of task IDs that must be completed before this task
- **`is_ready(task_id)`** - returns `True` if the task exists and all its dependencies are completed, `False` otherwise
- **`complete_task(task_id)`** - marks the task as done and makes dependent tasks potentially ready. If called on a non-ready task (pending or already completed), the operation should still mark it as completed (idempotent behavior)
- **`get_next()`** - returns the highest priority task that is ready (all dependencies completed), or `None` if no tasks are ready
- **`get_status(task_id)`** - returns "pending", "ready", or "completed."
- **`get_ready_tasks`** - returns a list of task IDs that are ready, sorted by priority (highest first)

Note: When multiple tasks have the same priority and are ready, return them in the order they were added (FIFO).

Example 1 Input:

```
9
add_task task1 10 []
add_task task2 5 []
add_task task3 15 ["task1"]
add_task task4 20 ["task1", "task2"]
is_ready task1
is_ready task3
get_next
complete_task task1
is_ready task3
```

Output:

```
True
False
task1
True
```

Explanation: task1 and task2 have no dependencies, so they're ready. task3 depends on task1, task4 depends on both task1 and task2. Initially, task1 is ready and has the highest priority among ready tasks (task1: 10, task2: 5). After completing task1, task3 becomes ready since its only dependency is satisfied.

Constraints * $1 \leq \text{number of operations} \leq 250$ * $1 \leq \text{length of task_id} \leq 50$ * $0 \leq \text{priority} \leq 1000$ * Number of dependencies per task ≤ 20 * Total number of unique tasks ≤ 200 * Task IDs contain only letters, numbers, and underscores * Dependencies must reference existing tasks (no circular dependencies in input)

Input Format for Custom Testing Input from stdin will be processed as follows and passed to the function:

The first line contains an integer n , the number of operations. Each of the next n lines contains a single operation in one of the following forms: * `add_task` — Add a task. Dependencies is a JSON array of task IDs, e.g., `[]` or `["task1", "task2"]`. * `is_ready` — Check if a task is ready (returns `True` or `False`). * `complete_task` — Mark a task as completed. * `get_next` — Get the next executable task (returns `task_id` or `None`). * `get_status` — Get the status of a task (returns `"pending"`, `"ready"`, or `"completed"`). * `get_ready_tasks` — Get all ready tasks as a comma-separated list sorted by priority (highest first), then FIFO for same priority.

Sample Case 0 Sample Input

```
9
add_task task1 10 []
add_task task2 5 []
add_task task3 15 ["task1"]
add_task task4 20 ["task1", "task2"]
is_ready task1
is_ready task3
get_next
complete_task task1
is_ready task3
```

Sample Output

```
True
False
task1
True
```

